

Mid-Project Update Report

Optimal Tic-Tac-Toe AI Agent Using Minimax and Alpha-Beta Pruning

CSE440 -- Sec-02 | Group 01 | Repository: cse440-tictactoe-ai-group01 | Covers Weeks 1–5

Project Overview

This project implements an intelligent AI agent that plays Tic-Tac-Toe optimally against a human opponent. The system is built around classical game-tree search: the Minimax algorithm for exhaustive exploration of possible move sequences, Alpha-Beta Pruning to eliminate branches that cannot influence the final decision, and heuristic evaluation functions to score non-terminal board states. Beyond producing an AI that never loses, the project also studies how different heuristic strategies affect decision-making speed and search efficiency.

Objectives

- Implement the classic Minimax algorithm for Tic-Tac-Toe.
- Optimize search using Alpha-Beta Pruning to reduce computation time.
- Design and compare multiple heuristic evaluation functions.
- Analyze performance gains achieved through pruning and heuristics.
- Deliver a playable interface (CLI first, GUI later) for human-vs-AI gameplay.

Progress Summary (Weeks 1–5)

All planned milestones for the first half of the project have been completed on schedule. The table below summarizes weekly deliverables and ownership.

Week	Milestone	Responsible	Status
Week 1	Repository setup, README, project structure	Sourav	Complete
Week 2	Game board implementation, CLI, win/draw detection	Sudipta	Complete
Week 3	Minimax algorithm implementation	Khatune Jannat	Complete
Week 4	Alpha-Beta Pruning optimization	Tabassum	Complete
Week 5	Full integration — playable Human vs. AI (CLI)	Sourav	Complete

Work Completed

Week 1 — Project Foundation:

- Initialized the GitHub repository (cse440-tictactoe-ai-group01) with the required course structure (main.py, support/, data/, others/).
- Drafted the README with project scope, objectives, and setup instructions.

Week 2 — Core Game Logic:

- Implemented the Tic-Tac-Toe board representation and state management in support/game.py.
- Built a command-line interface for move input and board display.
- Added win, loss, and draw detection logic, verified against all terminal board configurations.

Week 3 — Minimax Algorithm:

- Implemented the recursive Minimax algorithm in support/minimax.py.
- Validated correctness by confirming the AI always forces a win or draw, never a loss, across sample games.
- Measured baseline performance: full game-tree exploration from an empty board.

Week 4 — Alpha-Beta Pruning:

- Implemented Alpha-Beta Pruning in support/alpha_beta.py as an optimization layer over Minimax.
- Confirmed identical decision quality to plain Minimax while significantly reducing the number of nodes evaluated.

Week 5 — Integration:

- Integrated the game engine, Minimax/Alpha-Beta search, and CLI into a single playable Human-vs-AI experience via main.py.
- Conducted end-to-end testing across multiple play-throughs to confirm stability and correct game-ending behavior.

Preliminary Observations

- Alpha-Beta Pruning noticeably reduces the number of board states evaluated compared to plain Minimax, without changing the outcome of any game.
- The AI consistently achieves the theoretical optimum for Tic-Tac-Toe: it never loses, winning against suboptimal play and drawing against optimal play.
- Early experimentation suggests move-ordering and heuristic design will be the main levers for further speed improvements in Week 6 and beyond.

Challenges Encountered

- Ensuring the Alpha-Beta implementation preserved exact decision equivalence with Minimax required careful test-case comparison across many board states.
- Coordinating sequential dependencies between algorithm components (game logic → Minimax → pruning) required close team communication to avoid integration delays.

Next Steps (Week 6 and Beyond)

- Design and compare additional heuristic evaluation functions for non-terminal states.
- Benchmark and document performance differences between plain Minimax, Alpha-Beta Pruning, and heuristic-guided search.
- Begin development of a graphical interface (Pygame or Tkinter) to complement the existing CLI.
- Prepare the final report, final presentation, and demo video for project completion.

Team Contribution Summary

- Sourav — Repository setup and structure; full system integration (Weeks 1, 5).
- Sudipta — Game board, CLI, and win/draw detection logic (Week 2).
- Khatune Jannat — Minimax algorithm implementation (Week 3).
- Tabassum — Alpha-Beta Pruning optimization (Week 4).
- All members — Collaborative testing, review, and preparation of this report and presentation.